

Avaloka AI 1.0 — Technical User Guide

- [Avaloka AI 1.0 — Technical User Guide](#)
 - [About this guide](#)
 - [1. What Avaloka is](#)
 - [2. Getting started](#)
 - [3. Data capabilities](#)
 - [4. Machine learning](#)
 - [5. Visualization](#)
 - [6. Automation and continuity](#)
 - [7. Trust: how Avaloka checks itself](#)
 - [8. Integration capabilities](#)
 - [9. Prompting patterns](#)
 - [10. Editions](#)
 - [11. Troubleshooting](#)
 - [Appendix A — Capability to prompt map](#)
 - [Appendix B — Configuration reference](#)

Avaloka AI 1.0 — Technical User Guide

Document version 1.0 · Applies to Avaloka 1.6 (open-source release line)

About this guide

This guide is for the people who **use** Avaloka: analysts, data scientists, data engineers, and the platform teams who integrate it. It describes what the product can do, how to ask it for those things, and how to connect it to the systems you already run.

It is deliberately separate from `README.md` and `docs/architecture.md`. Those describe how Avaloka is *built* — the agent graph, the state object, the design decisions. This describes what Avaloka *does for you*, and assumes no interest in its internals.

If you want to	Read
Use Avaloka's capabilities	this guide
Follow a hands-on walkthrough of the UI	USER_GUIDE.md
Install it	INSTALL.md
Know what your edition includes	EDITIONS.md
Understand how it is built	architecture.md
Call it from code	api.md · cli.md
Deploy it on Kubernetes	deployment.md

How to read the prompt examples

Avaloka is driven by natural language. Every capability below is shown with the kind of prompt that invokes it. These are **patterns, not magic strings** — the phrasing is yours; the examples show the *shape* of a request that works, and what Avaloka does with it.

Convention used throughout: prompts appear as quoted user input, and the notable part of Avaloka's behaviour is described after them. Where a capability is gated by edition, that is stated inline.

1. What Avaloka is

Avaloka is an **AI-native data platform**. You point it at data and describe what you want in plain language; a team of specialised agents samples and profiles the data, plans the work, writes and validates the code, executes it — locally or on a distributed cluster — trains models, serves predictions, and keeps every artifact.

Three properties distinguish it from a chat assistant that writes code:

It runs the work, it does not just describe it. Generated code is validated against the real schema and executed; you receive results and artifacts, not a snippet to paste.

Every number is measured. Avaloka does not estimate figures it has not computed. If the evidence does not contain an answer, it says so rather than producing a plausible one.

Conclusions are checked, not just produced. An evidence layer screens data for leakage before training, evaluates models against a mandatory baseline, and checks the narrative against the computed numbers before you see it.

The unit of work: a thread

Work happens in a **thread** — a conversation bound to one or more datasets. Threads hold history, preferences, and produced artifacts, so later requests can refer to earlier ones (“the cleaned dataset from before”). Threads are listed, resumed, and deleted through the UI, the API, or the CLI.

2. Getting started

```
./scripts/install.sh           # open source
./scripts/install.sh --check   # what does my install resolve to?
```

Then either open the web UI, or drive it over HTTP:

```
curl -X POST $AVALOKA_API/threads
curl -X POST $AVALOKA_API/api/upload -F file=@sales.csv
curl -X POST $AVALOKA_API/threads/$THREAD/messages \
  -H 'content-type: application/json' \
  -d '{"message": "What is in this dataset?"}'
```

Full installation instructions, including edition licensing, are in [INSTALL.md](#).

3. Data capabilities

The thirteen capability areas below are the product’s functional surface. Each is exercised by an internal conformance suite of 281 prompts, so the examples here reflect requests Avaloka is tested against rather than aspirations.

3.1 Ingestion and catalog

Bring data in, and know what you have.

Local upload. Drop a file through the UI or `POST /api/upload` (**100 MB per file**; use a connection for larger data). Avaloka detects format and delimiter, samples the file, and registers it in the thread’s catalog under a name you can refer to later.

Register data already in place. For data that should not move, register it where it lives — `POST /api/register-existing-folder` for a directory, `POST /api/register-existing-storage` for an object-store prefix. Avaloka reads it in place; nothing is copied.

Multi-file upload. Files uploaded together are grouped, so a set of related extracts is one catalog entry rather than a scatter of UUIDs.

“Register my Postgres database” “Use the housing.csv I uploaded earlier” “List my datasets”

Supported formats

Category	Formats
Tabular text	CSV, TSV, delimited text (delimiter sniffed)
Spreadsheets	Excel (.xlsx , .xls)
Structured	JSON, XML
Columnar	Parquet, Avro
Table formats	Delta Lake, Apache Iceberg

Compressed variants of the text formats are handled transparently.

3.2 Discovery and profiling

Understand a dataset before doing anything to it. This is usually the first request in a thread, and Avaloka volunteers much of it unprompted.

“What’s in this dataset? Give me the schema with types and one example value per column.” “How many rows are there, and which columns have missing values and how many?” “What fraction of records have a cabin recorded?”

You get: row and column counts, per-column types, null counts and percentages, cardinality, distribution summaries for numeric columns, top values for categorical ones, and a data-quality report flagging high-missingness columns, constant columns, and duplicate rows.

Correlations are computed across numeric columns, so “what relates to what” is answerable immediately.

Profiling runs automatically on upload — sample, profile, then explain — and produces a **readiness briefing** before you have asked anything:

Element	What it gives you
Score and reasoning	A readiness score, and why it is what it is
Key Relationships	Column pairs expected to move together
Red Flags	Problems to know about <i>before</i> you trust a result
Quick Wins	Small clean-up steps worth doing first

This is the cheapest quality gate in the product: reading Red Flags before running an analysis prevents most of the results people later have to throw away.

3.3 Data transformation

Clean, reshape, and derive — described in business terms, not code.

“Fill the missing total_bedrooms with the column median and tell me how many rows you fixed.” “Add two engineered columns: rooms_per_household = total_rooms/households and bedrooms_per_room = total_bedrooms/total_rooms. Show 5 rows.” “Impute Age with the median age of each Pclass × Sex group, then report how many were filled and the new overall median.” “Drop rows where the target is null and tell me how many that removed.”

Transformations produce a **new named dataset** in the thread rather than mutating the source, so a later request can refer to “the cleaned dataset” and the original remains intact. Avaloka reports what it changed and how much — “filled 207 rows” rather than “done”.

The generated code is available: `GET /analysis/{id}/code`, and exporting it is open in every edition.

3.4 Analytics and statistics

Aggregate, rank, and quantify.

“Which 5 columns correlate most strongly with median_house_value? Values included.” “Average median_house_value by ocean_proximity, highest first, with row counts.” “Min, max, median, p90, p99 of median_income.” “Fraud rate by ProductCD, and tell me whether the difference is significant.”

Avaloka is built for **actionable intelligence, not just charts**: group-bys, distributions, percentiles, correlation structure, segment comparisons, and hypothesis tests. Where a statistical claim is made, the test and its assumptions are stated — an unsupported significance claim is flagged by the evidence layer before it reaches you (§7).

3.5 Fidelity and scale — sample versus full

This is the concept most worth understanding, because it governs both accuracy and cost.

For responsiveness, Avaloka profiles and explores on a **sample**. When a number must be exact, it runs against the **entire dataset**.

“How many distinct aisles are there?” → fast, sampled “Now compute that on the entire dataset.” → exact, full scan

Avaloka always tells you which it did. A figure computed on a sample is labelled as such. This matters: a sampled distinct-count is an estimate, and presenting it as exact would be a wrong answer delivered confidently.

Entire-dataset execution and distributed (Ray) execution are **cost-gated**: open on self-hosted installs where the compute is yours, capped on Avaloka-hosted plans. See [EDITIONS.md](#).

4. Machine learning

4.1 Model planning

Avaloka proposes before it trains. Ask for a model and you get a plan — task type, features, target, preprocessing, split strategy, and evaluation approach — which you can amend in conversation.

“Set up a model to predict median_house_value from median_income, housing_median_age, total_rooms, population. 3 epochs. Plan only — don’t train yet.” “I want to predict Survived using Pclass, Sex, Age and Fare. Show the plan first.” “Add SibSp as a feature too.” “Change it to 5 epochs.”

The plan is a conversational object. Amending it does not restart the thread.

4.2 Training execution and the model registry

“Looks good — go ahead and train it.” “Change it to 5 epochs and train it now.”

Training runs locally or on a Ray cluster depending on size and configuration. Before a model is fit, the **integrity screen** runs (§7.1); if it finds target leakage, training is blocked rather than producing a model with an excellent, meaningless score.

Every run is registered: `GET /api/models` lists them, `GET /api/models/{run_id}` returns metrics, parameters, and artifacts. Results include cross-validated metrics and a comparison against a mandatory trivial baseline — a model that does not beat the baseline is reported as such.

ML training and inference are commercial capabilities.

4.3 Inference and serving

“Deploy this model for predictions.” “Score these 500 rows against the model I trained.” “Stop the inference service.”

`POST /api/models/{run_id}/configure-inference-service` stands up an endpoint; `POST /api/models/{run_id}/inference` scores records; `POST /api/models/{run_id}/stop-inference-service` tears it down. Serving scales to zero when idle so a deployed endpoint does not bill continuously.

5. Visualization

“Histogram of median_income with a sensible bin count.” “Bar chart: survival rate by Pclass.”
“Heatmap of order volume: order_dow vs order_hour_of_day.”

Chart type, binning, and axes are inferred from the data and the question. Visuals are artifacts of the thread, retrievable via `GET /api/assets/{session_id}`.

Charts also appear **without being asked for**: every upload and every analysis that produces a dataset refreshes an automatic set of three to five charts chosen to explain the data in front of you. Asking a chart-shaped question steers what gets rendered.

6. Automation and continuity

6.1 Scheduling

“Every Monday 9am, recompute the fraud rate by ProductCD and store the result.” “List my scheduled tasks.” “Stop the weekly fraud job.”

Scheduled work is managed through `GET /tasks`, `GET /tasks/{id}/status`, `GET /tasks/{id}/runs`, and `DELETE /tasks/{id}`. Each run is retained, so a scheduled analysis has history rather than only a latest value.

Scheduling is cost-gated; scheduled delivery of results is commercial.

6.2 Memory, preferences, and history

Avaloka remembers within a thread.

“Preference: always include row counts in any table you show me.” “What did we find about fraud rates in earlier analyses?” “Use the cleaned dataset from before.”

Stated preferences persist and are applied to later answers. Prior findings are retrievable, so a thread accumulates context rather than restarting each turn.

Four things are retained: your **dataset’s structure**, so you do not re-describe it each prompt; your **working style**, accumulated quietly as you work (that you prefer medians, say, or always exclude nulls); **anything you explicitly ask it to remember**; and your **past results**, which can be reused rather than recomputed.

Explicit memories are **protected — newer memories never crowd them out**. A preference you stated once holds for the thread rather than decaying as context fills.

6.3 Plan graph and job summaries

Before running a multi-step job, Avaloka can render the execution plan as a graph — `GET /threads/{thread_id}/planner-graph` — so you can inspect the path before committing to it. After a job, a summary describes what ran, what it produced, and what it cost.

7. Trust: how Avaloka checks itself

An analytics platform that is confidently wrong is worse than one that fails loudly. Three mechanisms run automatically.

7.1 Integrity screening (before training)

Six checks run before a model is fit:

Check	Catches
Target in features	The target column left among the inputs
Target correlation ≥ 0.98	A feature that is a near-copy of the target
Duplicate rows across splits	Memorisation scoring as generalisation
Identifier-like features	High-cardinality keys that will not exist for new entities
Temporal leakage	Training on rows that postdate the test period
Constant / near-constant features	No signal, distorted importance

Findings are reported with a `safe_to_train` verdict. Target leakage is the most common silent failure in automated data science: a leaking model reports excellent metrics and fails in production.

7.2 Evaluation with a mandatory baseline

Every model is cross-validated with the splitter the data actually requires — time series get `TimeSeriesSplit`, grouped data gets `GroupKFold` — and the choice is recorded with its reason. Every result is scored against a trivial baseline, and **“beats baseline” requires clearing fold-to-fold noise**, not merely a higher mean.

This is why 94% accuracy is not automatically good news: if 95% of rows are one class, a majority-class predictor does better, and Avaloka says so.

7.3 Claim verification (before you see the answer)

The narrative itself is checked against the computed evidence for causal overreach, invented numbers, contradictions with the metrics, absolute language, and unsupported significance claims. The check is

deterministic — no model is asked to grade another model’s honesty, so the check cannot itself hallucinate.

8. Integration capabilities

8.1 Data sources

Type	Supported
Object storage	Amazon S3, Google Cloud Storage, Azure Blob Storage
Databases — query and register	PostgreSQL, MySQL, SQLite, SQL Server, Oracle, MariaDB
Databases — transfer target	PostgreSQL, MySQL
Local	Filesystem paths, direct upload (100 MB per file)
URLs	Direct HTTP(S) sources

Registering a database and transferring into it are different capabilities. Six engines can be registered and queried; transfers currently target PostgreSQL and MySQL. Plan a migration around the second list, not the first.

Connections are registered once and referred to by name:

POST /api/database/connect	register a database connection
POST /api/v1/database/query	query it directly
POST /api/database/tables-to-analysis	pull tables into a thread
GET /buckets/list	list object-store buckets

Credentials are encrypted at rest (POST /api/mcp-connections/{id}/encrypt · .../decrypt) and redacted from logs and generated code.

Database and cloud connectors are commercial capabilities; file connectors are open in every edition.

8.2 The Data Transfer Agent

Move data between systems by describing the move. Four directions are supported: cloud-to-cloud, cloud-to-database, database-to-cloud, and database-to-database.

“Transfer aisles.csv from the instacart-mba connection to the walmart connection under transfers/aisles_copy.csv.” “Move my cleaned housing dataset to the nyc-taxi bucket connection.”

“Load the customers table from Postgres into the analytics warehouse.”

The agent deduces schema, handles format conversion, and validates the transfer. Large transfers run distributed. Cross-account and cross-cloud moves are supported where both connections are registered.

Reads and writes are not symmetric, and this catches people out:

Direction	Files	Databases
Read	CSV, JSON, Parquet	MySQL, PostgreSQL
Write	CSV, TSV, JSON, Parquet, Excel (.xlsx), XML, Avro, ORC	MySQL, PostgreSQL

Avaloka writes eight file formats and reads three. CSV and Parquet stream directly from a cloud bucket; **JSON must be loaded into Avaloka first** — if your JSON sits in a bucket, convert it to CSV or Parquet before transferring. To move data out of a format that is not readable, load it into a thread first, then transfer from there.

All eight write formats are available when the destination is a cloud bucket.

8.3 HTTP API

47 endpoints across threads and messaging, datasets and upload, analysis and code, assets, models and inference, tasks and scheduling, connections, and health. Full reference: [api.md](#).

The essential loop:

POST /threads	create a thread
POST /api/upload	add data
POST /threads/{id}/messages	ask
GET /api/assets/{session_id}	collect outputs

8.4 Command line and MCP

The **CLI** drives missions from a terminal — planning, budgets, and execution without a browser. See [cli.md](#).

The **MCP server** exposes Avaloka as a tool provider to MCP-compatible clients, so an external agent or IDE can use Avaloka’s data capabilities directly.

8.5 Deployment targets

Target	Use
Local / kind	Laptop, CI
Google GKE	Production, with Ray/KubeRay
Amazon EKS	Production, with Ray/KubeRay
Azure AKS	Production, with Ray/KubeRay

Set `CLOUD_PROVIDER` and that cloud's credential variables; nothing about a particular account is compiled in. See [deployment.md](#).

8.6 Model providers

Avaloka is not tied to one LLM vendor. Groq, OpenRouter, OpenAI-compatible endpoints, AWS Bedrock, GCP Vertex AI, Azure AI, and **in-cluster or local models** (vLLM, Ollama) are all selectable by configuration. If a provider fails or deprecates a model, Avaloka falls back automatically — including to a local model sized for the machine it is running on, so an outage degrades service rather than stopping it.

9. Prompting patterns

Avaloka responds to intent, not syntax. These patterns generalise across capabilities.

Be specific about the output you want. “Average revenue by region, highest first, with row counts” gets you exactly that. “Tell me about revenue” gets an exploration.

Ask for a plan when the work is expensive. Adding “plan only — don't run it yet” makes Avaloka propose before spending. Useful before training or a full scan.

Say when you need exactness. Sampled answers are fast; “on the entire dataset” makes them exact. Avaloka labels which it used either way.

Refer to earlier work. “The cleaned dataset from before”, “the model we trained” — thread memory resolves these.

State preferences once. “Always include row counts” applies to the rest of the thread.

Ask for the code. “Show me the code you ran” returns it; exporting generated code is open in every edition.

Disagree with it. If a result looks wrong, say so. Avaloka is built to hold its position when the evidence supports it and to correct itself when it does not — it will not simply agree with you.

Questions worth asking that people forget

“Is this model actually better than guessing?” “Was that computed on a sample or the whole dataset?” “What’s the row count behind that percentage?” “Are there any data quality problems I should know about before I trust this?”

10. Editions

Two independent questions decide what you get:

Question	Values	Decides
Where does it run?	self-hosted · Avaloka-hosted	who pays for compute
What did you buy?	OSS · Free · Professional · Enterprise	which capabilities are licensed

A capability is unavailable for exactly one of three reasons:

Reason	Meaning	Workaround
OPEN	Not gated	—
COST	We provide the compute, so hosted plans are capped	Self-host
COMMERCIAL	Implementation is not in the OSS distribution	None

Open in every edition: data analysis, generated-code export, local execution, file connectors.

Cost-gated (open when self-hosted): distributed Ray execution, entire-dataset scans, scheduling, batch jobs.

Commercial: database connectors, cloud connectors, cloud provisioning, ML training, ML inference, scheduled delivery, team collaboration, analysis sharing and comments, notifications, swarm intelligence, SSO and audit.

`./scripts/install.sh --check` reports what your install resolves to. Full matrix: [EDITIONS.md](#).

11. Troubleshooting

Symptom	Cause	Resolution
A capability is refused	Edition gate	Run <code>install.sh --check</code> ; if <code>COST</code> on a self-hosted install, that is a bug — please report it
A number differs between two answers	One was sampled, one was full	Ask “on the entire dataset” for the authoritative figure
Training is blocked	Integrity screen found leakage	Read the findings; a leaking feature usually should be dropped, not overridden
A model reports poor results	It did not beat the baseline	This is a real finding, not a failure — the signal may not be in the data
Startup fails naming a variable	Cloud identity is unconfigured	Set the named variable; Avaloka does not assume a project or registry
A provider returns 404 for a model	Model deprecated upstream	Fallback should engage automatically; verify with <code>scripts/ops/verify_models.py</code>

Appendix A — Capability to prompt map

#	Capability	Example prompt
1	Ingestion & catalog	“Register my Postgres database”
2	Discovery & profiling	“What’s in this dataset? Schema with types and an example value per column.”
3	Transformation	“Fill missing total_bedrooms with the column median and tell me how many rows you fixed.”
4	Analytics & statistics	“Average median_house_value by ocean_proximity, highest first, with row counts.”
5	Model planning	“Predict Survived using Pclass, Sex, Age, Fare. Show the plan first.”
6	Training & registry	“Looks good — train it.”
7	Inference & serving	“Deploy this model for predictions.”
8	Data transfer	“Transfer aisles.csv from the instacart connection to the walmart connection.”
9	Visualization	“Heatmap of order volume: order_dow vs order_hour_of_day.”
10	Scheduling	“Every Monday 9am, recompute the fraud rate by ProductCD.”
11	Fidelity & scale	“Now compute that on the entire dataset.”
12	Memory & preferences	“Always include row counts in any table you show me.”
13	Plan graph & summaries	“Show me the plan before you run it.”

Appendix B — Configuration reference

Variable	Purpose
CLOUD_PROVIDER	local gcp aws azure
CONTAINER_REGISTRY	Override the derived image registry
GCP_PROJECT_ID · GOOGLE_APPLICATION_CREDENTIALS	GCP identity
AWS_ACCOUNT_ID · AWS_REGION	AWS identity
AZURE_SUBSCRIPTION_ID · AZURE_LOCATION · AZURE_CONTAINER_REGISTRY	Azure identity
AVALOKA_DEPLOYMENT_PROFILE	laptop cluster — sizes the local model tier
AVALOKA_EDITION · licence key	Edition selection
DEFAULT_SAMPLE_MAX_ROWS	Sampling ceiling for exploration

Provider and model variables are documented in `.env.example`.

Avaloka is open source under the Apache License 2.0. Corrections and additions to this guide are welcome — see [CONTRIBUTING.md](#).